Simulation II

Topics

1. Understand methods to generate random observations from probability distribution
2. Simulation optimization combines simulation with optimization techniques.
3. Outline of a Simulation Study

Difference Between Pseudo Numbers and Uniform Random Numbers

Feature	**Pseudo-Random Numbers** **The word “pseudo” means “fake or false” or “not genuine, but resemble the real thing.”**	**Uniform Random Numbers**
Format	Integer (e.g., 3, 7, 0)	Real number (e.g., 0.333, 0.778)
Range	0 to m−1	0.0 to 1.0
Generated By	Algorithms (linear congruential)	Scaling pseudo-random integers
Nature	Deterministic (reproducible), if seed values is used at the beginning	Appears continuous, [0, 1]

Generation of Random Observations from a Probability Distribution

Objectives of this section

- Understand the role of random observations in simulation.
- Learn key methods for generating random observations from various probability distributions.

Key Questions

- Why generate random observations?
- What are the common methods?
- How are they applied in simulation?

Why Generate Random Observations?

- Simulations mimic real-world stochastic systems by generating events (e.g., arrivals, service times).
- Random observations from specific probability distributions ensure realistic event generation.
- Example: Simulating customer interarrival times using an exponential distribution.
- *Note: A stochastic system is a system whose behavior involves randomness or uncertainty, meaning you cannot predict its exact outcome, only the probability of different outcomes.*

Overview of Methods

- Following are some key concept we will learn.
  - Inverse transformation method.
  - Handling discrete and continuous distributions.
  - Special cases: Exponential, Erlang, Normal, and Chi-Square distributions.
  - Acceptance-rejection method.

Simple Discrete Distributions

- Allocate uniform random numbers to outcomes based on probabilities.
- Example: Coin flip simulation:
  - Heads and Tails

IF(RAND() < 0.5, "Heads", "Tails")
Heads
Tails
Tails
Heads
Tails
Tails
Tails
Heads
Tails
Heads
Heads
Tails
Tails

Linking Uniform Random Numbers to Other Distributions?

- Uniform Random Numbers (RAND()) are Simple:
  - Excel's RAND() generates numbers between 0 and 1 where every value is equally likely.
  - Example: 0.2, 0.5, 0.99 all have the same probability.
- But Real-World Data Isn't Always Uniform:
  - Service times might follow an exponential distribution.
  - Sums of tasks might follow an Erlang distribution.
  - Heights might follow a normal distribution.

Inverse Transformation Method

- Step 1: Generate a uniform random number $r \in [0,1]$
- Step 2: Set $F(x) = r$ and solve for x
- Works for both discrete and continuous distributions
- Used for Exponential, Uniform, and other invertible distributions.

Step by Step Intuition

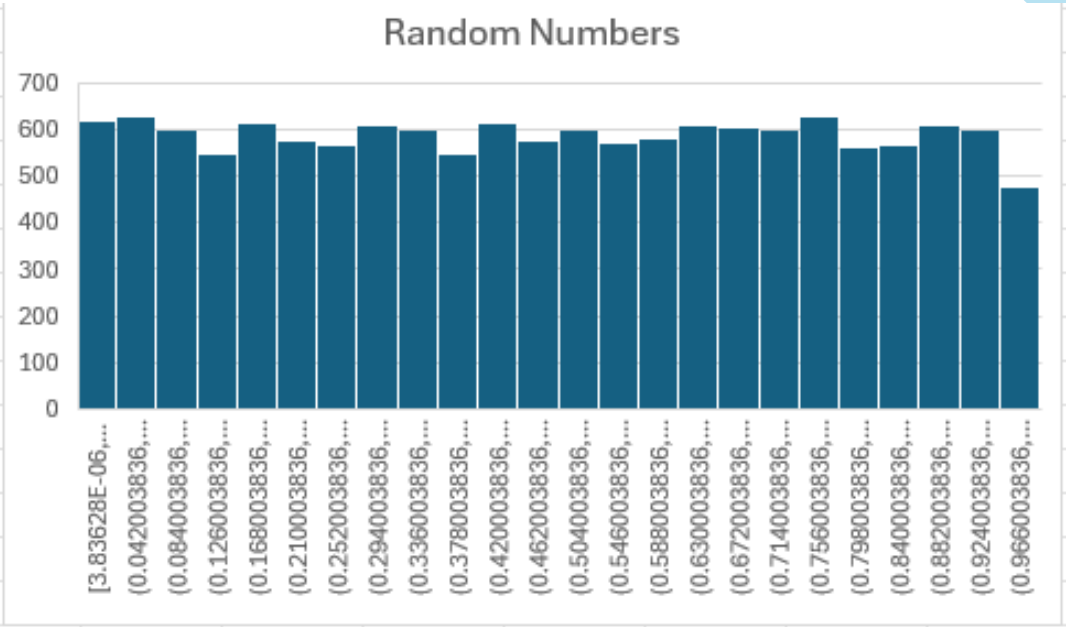
- Start with uniform (0,1) Randomness
- Reshape uniform to Target Distribution

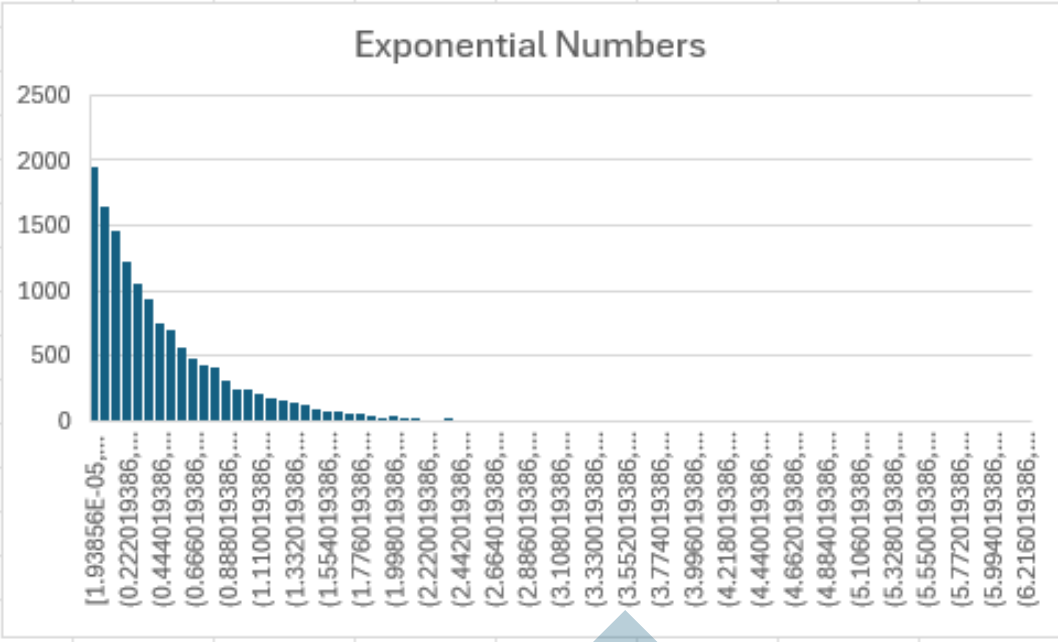
Exponential vs. Uniform in Excel

Formula	What it Does	Output Distribution
=RAND()	Raw uniform Randomness	Flat (0 to 1) Histogram will look flat
= - LN(RAND())/2	Transformed to exponential	Skewed (many small values, few large) Histogram will show exponential decay

A = RAND()
B = - LN(A)/2

A	B
0.229707034	0.735475275
0.267420163	0.659467106
0.984409882	0.007856461
0.506283416	0.340329327
0.505718749	0.340887299
0.2282115	0.738741225
0.555557602	0.293891491
0.70414354	0.175386526
0.446268199	0.403417582
0.641025843	0.222342753
0.381493605	0.481830595
0.877634584	0.065262482
0.183125957	0.848790536
0.290098477	0.618767419
0.827728581	0.09453499
0.896970148	0.054366349
0.648373582	0.216644117
0.104436157	1.129589665
0.612448275	0.245145395
0.747353959	0.145608183
0.26639334	0.66139067
0.713304992	0.168923096
0.030704216	1.741677646
0.797820361	0.112935909
0.286227355	0.625484418
0.810920643	0.10479254
0.599720707	0.25564561
0.38009706	0.483664319
0.321023488	0.568120494
0.267804508	0.658749005





Why Not Use Uniform Directly?

- Real-world processes (e.g., call center wait times) are rarely uniform.

- Using the wrong distribution in simulations gives misleading results.

In other words (Why Not Use Uniform Directly?)

How do we generate realistic random values?

- Computers generate:
  - Random numbers between **0 and 1 only**
- But real-world variables are:
  - Waiting time (Exponential)
  - Demand (Normal)
  - Failures (Gamma)

Problem: How do we move from simple random numbers to real-world randomness?

The Starting Point

Uniform Random Numbers

$$U \sim U(0, 1)$$

- Equal chance of any value between 0 and 1
- Examples:
  - 0.12, 0.56, 0.91

This is the only type of randomness computers naturally generate

The Main Idea

Transformation

We don’t generate new randomness—we **transform it**

Convert:

- From: Uniform random number U
- To: Desired random variable X

The General Formula

$$X=F^{-1}(U)$$

Where:

- U : uniform random number
- F^{-1} : inverse of cumulative distribution function
- X: required random value

Visualization of Concept

- Show graph:
  - CDF curve
  - Pick random U on vertical axis
  - Project to curve → find corresponding

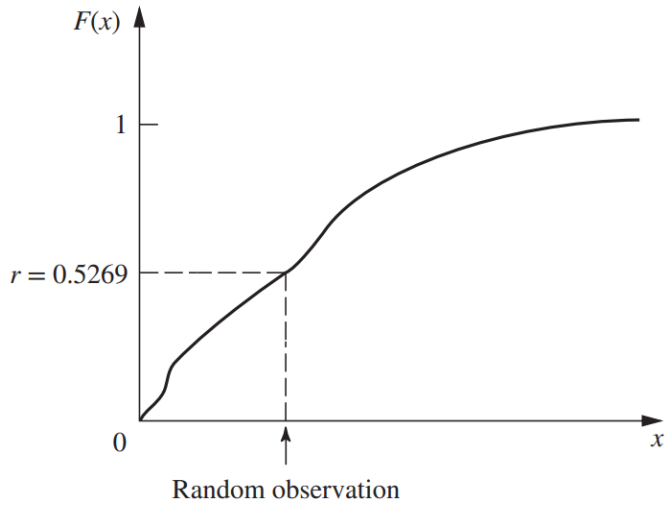


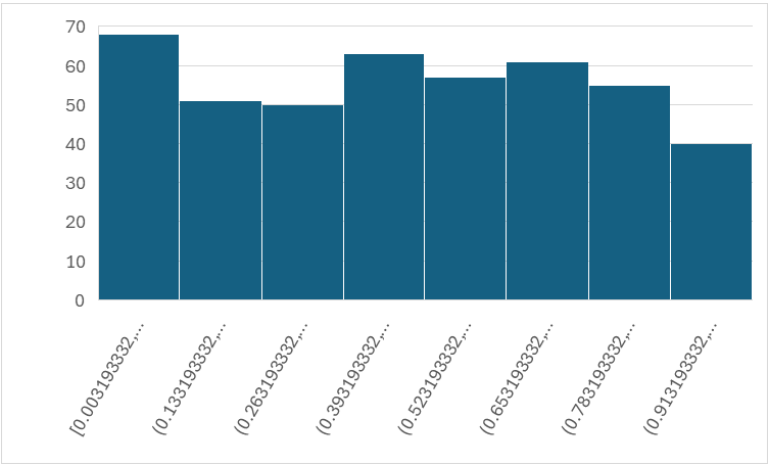
FIGURE 20.5
Illustration of the inverse transformation method for obtaining a random observation from a given probability distribution.

Simulation Workflow

- Collect and preprocess data → Fit distributions.
- Generate pseudo-random numbers (LCM).
- **Convert to (U(0,1) **.
- Transform to target distributions (inverse transform).
- Build the simulation model (logic, rules, animation).
- Run experiments (warm-up, replications).
- Analyze output (stats, sensitivity tests).
- Validate and report results.

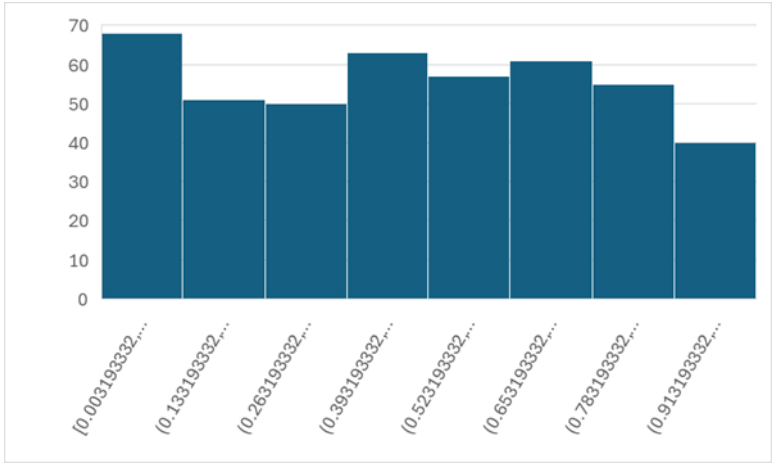
Inverse Transformation Explanation

- Turn a uniform random number (0 to 1) into a random observation from any desired distribution using its Cumulative Distribution Function (CDF).
- **Step-1: Understand CDF (Cumulative Frequency Distribution)**
  - The CDF, F(x), tells you the probability that a random variable X is less than or equal to x.
  - **Example:** For a uniform distribution between *a* and *b*:
  - F(x) = x−a/ b−a (a straight line from 0 to 1)



Step 2: Generate a Uniform Random Number

- Let r = random number between 0 and 1 (e.g., from RAND() in Excel).
- We call it uniform because every number in the range has equal probabilities and when we get a large number of data set each number will have equal frequency distribution.

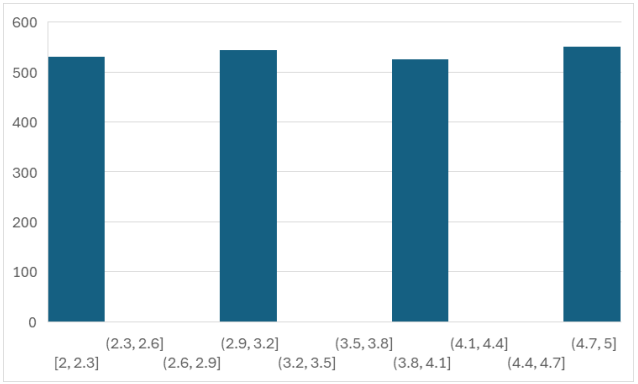


Step 3: Invert the CDF

- **Step 3: Invert the CDF**
  - Solve $F(x) = r$ for x:
  - $x = F^{-1}(\mathrm{r})$
  - This x is your random observation from the desired distribution!

Example: Uniform Distribution

- Generate random numbers between *a*=2 and *b*=5.



- **CDF:**

$$F(x) = \frac{x-2}{5-2} = \frac{x-2}{3}$$

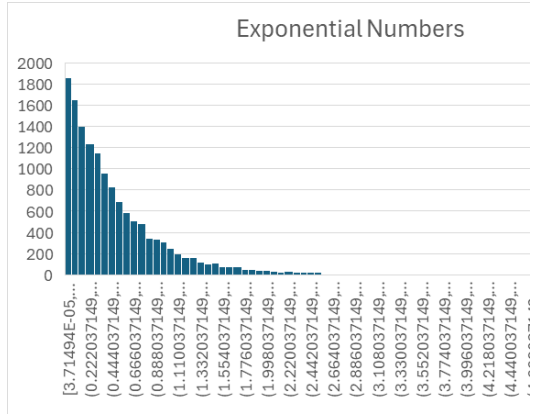
- **Invert Transformation:**

$$r = \frac{x-2}{3} \implies x = 2 + 3r$$

- Generate r:
  - If r = 0.04 then x = 2+3(0.4) = 3.2
  - If r = 0.8, then x = 2+3(0.8) = 4.4

Exponential Distribution

- Generate random observations from $f(x) = 2e^{-2x}$ (mean1/2)
- CDF $$F(x) = 1 - e^{-2x}$$



- Invert it: $$r = 1 - e^{-2x} \implies x = -\frac{\ln(1-r)}{2}$$
- Since $1 - r\ is\ also\ uniform, you\ can\ simplyfy\ to\ x = -\frac{\ln(r)}{2}$

Erlang Distribution

- Sum of k exponential random variables.
- $Generate\ k\ uniform\ number\ r_1, r_2, \dots, rk$
- $x = -\frac{1}{ka} \ln(r_1, r_2, \dots rk)$

Normal Distribution

- Central Limit Theorem Approach:
  - Sum 12 uniform numbers, subtract 6 → Approximate *N*(0,1).
  - Select: $\mu + \sigma \,.\, (\sum_{i=1}^{12} r_i \, - 6)$
- **Exact Methods:**
- Box –Muller transform
- Excel: = NORM.INV(RAND(), μ , σ

Bank Queue Simulation Example (Waiting Time & Service Time)

- This example simulates a simple single-server bank queue (like one teller serving customers). We'll model:
- Customer arrivals (random, exponentially distributed)
- Service times (random, following an Erlang (Gamma) distribution)
- Waiting times (based on queue dynamics)

Define Parameters

Parameter	Value	Explanation
Arrival rate (λ)	4 customers/hour	Average arrivals per hour
Service rate (μ)	5 customers/hour	Average service rate (Erlang-distributed)
Simulation time	8 hours	Bank operating hours
Shape (k) for Erlang	2	Service time follows Erlang(k=2, μ=5)

Generate Random Data

Customer	Interarrival Time (Exp)	Arrival Time	Service Time (Erlang)	Start Time	End Time	Waiting Time
1	=-LN(RAND())/4	=B2	=GAMMA.INV(RAND(), 2, 1/5)	=C2	=D2+E2	=0 (first customer)
2	=-LN(RAND())/4	=C2 + B3	=GAMMA.INV(RAND(), 2, 1/5)	=MAX(F2, C3)	=D3+E3	=D3 - C3

	A	B	C	D	E	F	G
1	Customer	Interarrival Time (Exp)	Arrival Time	Service Time (Erlang)	Start Time	End Time	Waiting Time
2	1	0.660272152	0.660272152	0.079716605	0.660272152	0.739988756	=0 (first customer)
3	2	0.072325272	0.732597424	0.984660878	0.739988756	1.724649635	0.252063455
4	3	0.140953523	0.140953523	0.1120424	0.140953523	0.252995923	-0.028911124
5	4	0.030925764	0.171879287	0.341121751	0.252995923	0.594117674	0.169242465
6	5	0.147546753	0.147546753	0.483283987	0.147546753	0.630830739	0.335737234
7	6	0.237144063	0.384690816	0.349994919	0.630830739	0.980825658	-0.034695897
8	7	0.853492539	0.853492539	0.199341284	0.853492539	1.052833823	-0.654151256
9	8	0.033973431	0.887465971	0.266431019	1.052833823	1.319264843	-0.621034952
10	9	0.267864746	0.267864746	1.097461153	0.267864746	1.365325899	0.829596407
11	10	0.107268365	0.375133111	0.411792399	1.365325899	1.777118298	0.036659287
12	11	0.371504691	0.371504691	1.013922351	0.371504691	1.385427043	0.64241766
13	12	0.033961967	0.405466658	0.515218061	1.385427043	1.900645103	0.109751403
14	13	0.072578628	0.072578628	0.297000751	0.072578628	0.369579379	0.224422123

Analyze Results

- Average Waiting Time
- Maximum Waiting Time
- Server Utilization

Simulation Optimization

Introduction to Simulation Optimization

- Simulation optimization combines simulation with optimization techniques to find the best system configuration under uncertainty.
- Key Goal:
  - Identify the optimal design or policy from a set of alternatives with high probability.
- Applications:
  - Inventory management
  - Queueing systems.
  - Manufacturing process design.

Why Use Simulation Optimization?

- Challenges of Simulation Alone
  - Provides statistical estimates, not exact solutions.
  - Compares alternatives but does not inherently optimize.
- Simulation Optimization Addresses
  - Efficiently searches for the best configuration.
  - Reduces computational effort by focusing on promising solutions.

Key Components of Simulation Optimization

- State Space
  - Small: Few configurations (e.g., comparing 5 inventory policies).
  - Large Discrete: Many configurations (e.g., server allocations in a network).
  - Continuous: Infinite configurations (e.g., tuning a parameter like reorder point).
- Performance Measures:
  - Mean response time, cost, etc.
- Optimization Techniques:
  - Ranking and selection, random search, gradient-based methods.

Ranking and Selection for Small State Spaces

- Bechhofer's Procedure:
- Example:
  - 5 populations with means μ_1 to μ_5.
  - Simulate N runs per population, compare sample averages.

Fully Sequential Procedures (Paulson/KN)

- Advantage:
- Reduces unnecessary sampling by discarding inferior options early.
- Steps:
  - Simulate all systems initially.
  - Eliminate systems unlikely to be optimal.
  - Focus sampling on remaining candidates.
- Use Case:
- When simulation runs are expensive (e.g., complex manufacturing systems).

Outline of a Major Simulation Study

Key Steps for Effective Simulation in Operations Research

Introduction

- Objective:
  - Understand the structured approach to conducting a simulation study.
- Importance:
  - Ensures accuracy, efficiency, and actionable results.
- Applications:
  - Manufacturing, healthcare, logistics, etc.

Step 1 – Formulate the Problem and Plan the Study

- Key Actions:
  - Define the problem with stakeholders.
  - Set objectives (e.g., reduce wait times, optimize inventory).
  - Identify specific issues and constraints (time, budget).
- Output: Clear project scope and performance measures

Step 2 – Collect Data and Formulate the Model

- Data Needs:
  - Probability distributions (e.g., interarrival/service times).
  - System rules (e.g., queue discipline).
- Model Components:
  - Flow diagrams, event definitions, state variables.
- Example: Queueing system with arrival/service distributions.

Step 3 – Validate the Model

- Methods:
  - Structured Walk-Through: Engage experts to review assumptions.
  - Comparison: Match model outputs with real-world data or analytical results (if available).
- Goal: Ensure the model mirrors the actual system.

Step 4 – Select Software and Construct the Program

- Software Options:
  - Spreadsheets (Excel): Simple models (e.g., coin-flipping game).
  - General-Purpose (Python/R): Flexibility but requires coding.

Step 5 – Test Model Validity

- Techniques:
  - Field Tests: Small-scale prototypes.
  - Sensitivity Analysis: Vary inputs to check output consistency.
  - Animation: Visual checks for logical flow.
- Example: Compare simulated vs. actual queue lengths.

Step 6 – Plan Simulations

- Design Decisions:
  - Run Length: Balance precision vs. computational cost.
  - Replications: Ensure statistical significance (e.g., 10,000 runs).

Step 7 – Conduct Runs and Analyze Results

- Process:
  - Execute simulations for each system configuration.
  - Record performance measures (e.g., average wait time).
- Output Analysis:
  - Point estimates and 95% confidence intervals.
  - Compare alternatives.
- Optimization: Use simulation-optimization if needed

Step 8 – Present Recommendations

- Written Report: Methodology, validation, results.

Best Practices

- Best Practices:
  - Involve stakeholders early.
  - Use variance-reduction techniques.
  - Document assumptions rigorously.

Conclusion

- Simulation studies require a structured, iterative approach.
- Combines statistical rigor with practical problem-solving.
- Future Trends: Integration with AI, real-time simulation.

Thanks